# FA 1

*Dela Cruz, Cloie Sofia R.  |  Zabala, Ralf Cedric T.*

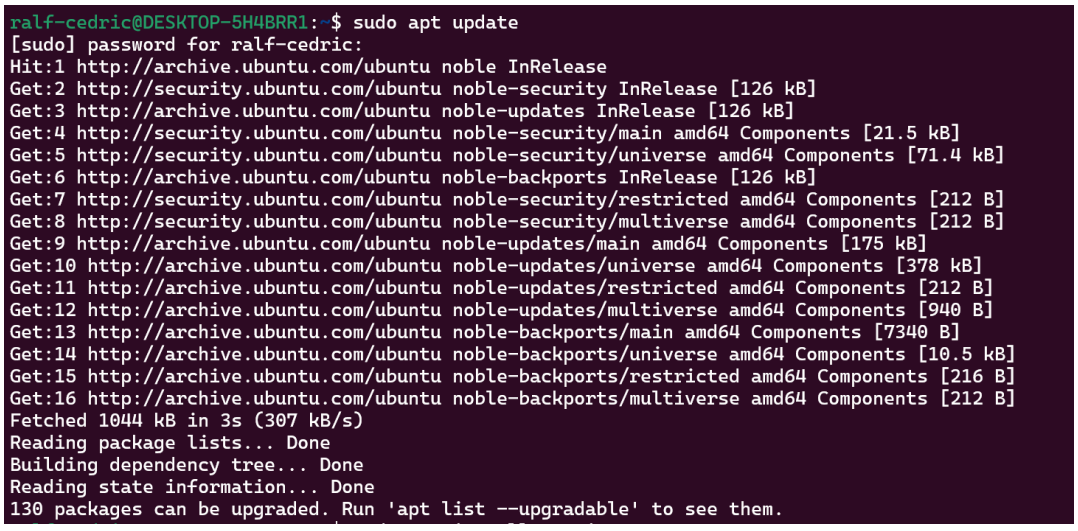
## I. Installing GIT on Ubuntu

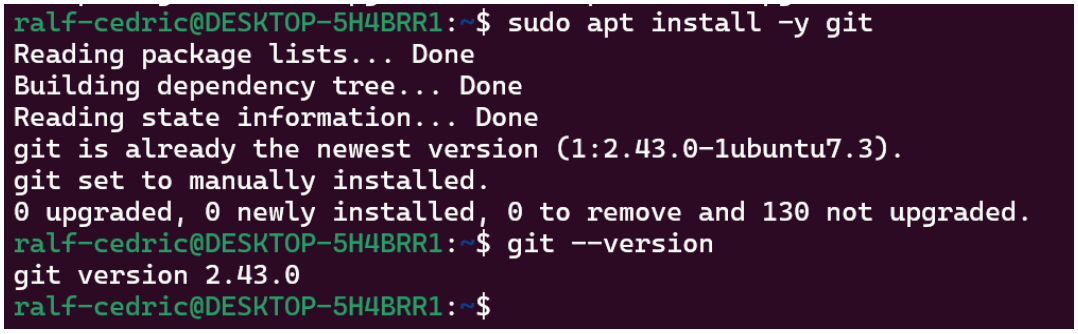
Today, we will be getting a sample code from GitHub to run a neural network. To accomplish this, we will be installing Git. This allows us to clone and manage the project source code from a GitHub repository (Git, 2025).

Run:

```
sudo apt update
sudo apt install -y git
git --version
```

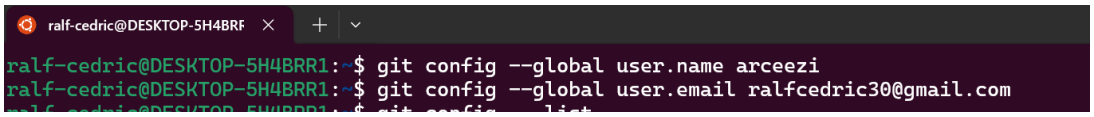
This updates everything and installs git on Linux Ubuntu. The Git version can also be checked using the command above. Take the images below for reference:



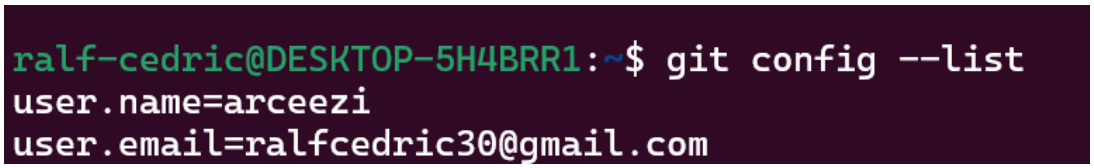
As shown above, the installed version is 2.43. This is done to ensure that Git is truly installed and available. Next, we can try to set a global username and email with this:

```
git config --global user.name <username>
git config --global user.email <email>
```




In this example, `<username>` used was *arceezi,* and `<email>` used was *ralfcedric30@gmail.com*. This lists the users on the system:

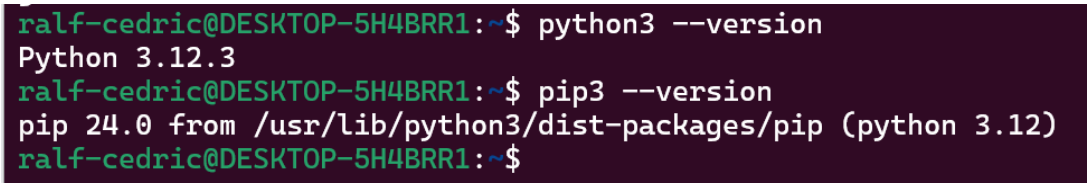
```
git config -list
```



# II. Checking the Python environment through Python & pip

Let’s also check the Python environment. To do this, run:

```
python3 --version
pip3 --version
```




This verifies the Python environment and helps us prepare for the next step. If either of the two is missing, install it.

For Python:

```
sudo apt update
sudo apt install -y python3
```

For pip:

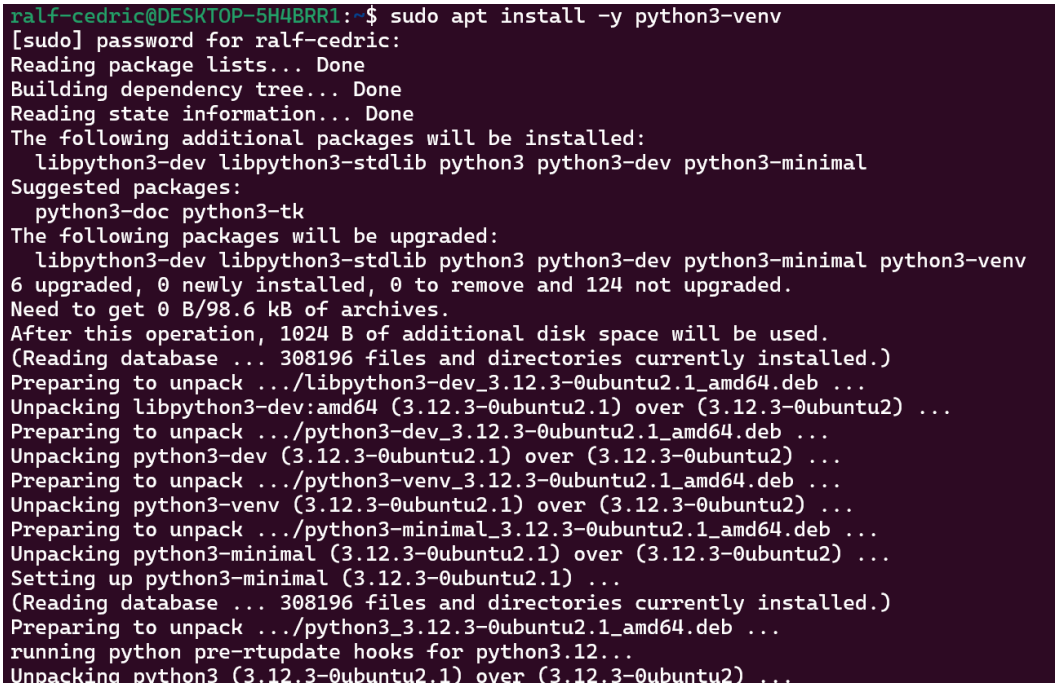
```
sudo apt install -y python3-pip
pip3 --version
```

Python will be our language for implementing our neural network, and pip is its package installer. This will be essential for installing our libraries later (Pip, 2008).

# III. Creating the *neural_network* Environment

In creating an environment, we first need to install the Python virtual environment, which is a lightweight and isolated directory for Python projects (Python Software Foundation, 2019). This is for one-time installation only. Do this with:

```
sudo apt install -y python3-venv
```




Now, we create the required environment: *neural_network*. Create a folder, and in this case, the name of the folder will be '*fa1_nn*' to match the activity's name. Name it however you like.

Run:

```
mkdir -p ~/fa1_nn
cd ~/fa1_nn
```

Navigate inside the created directory in the clean working folder. Then, create the environment using:

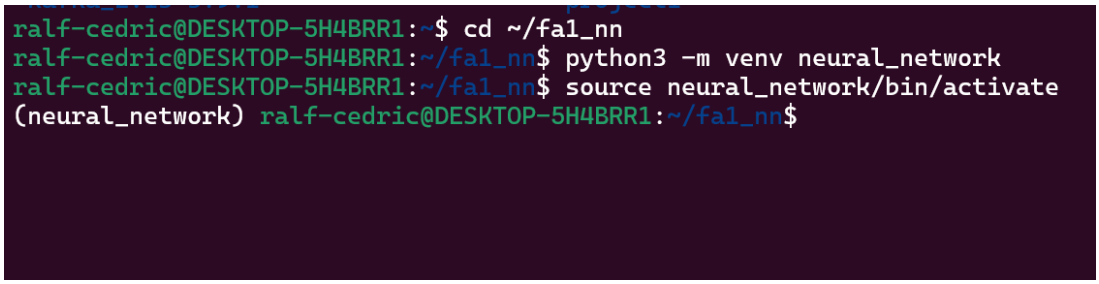
```
python3 -m venv neural_network
```




Then, activate the environment with the same environment name that you have created. In this case, *'neural network'* is the name of the environment. So, activate it with:

```
source neural_network/bin/activate
```

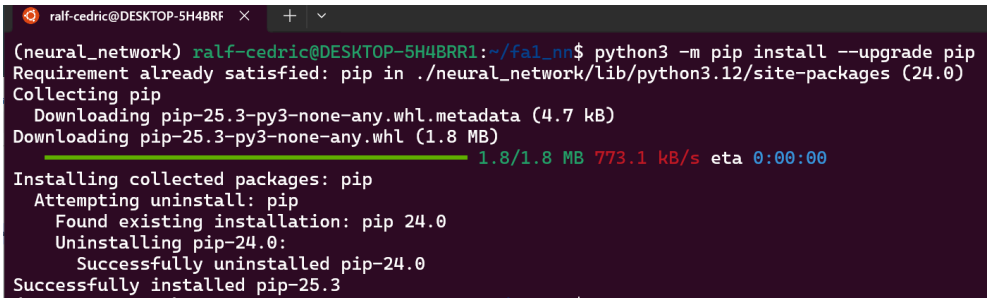
There would be an indication that you are inside the environment. The terminal should now look like:



# IV. Installing the necessary libraries

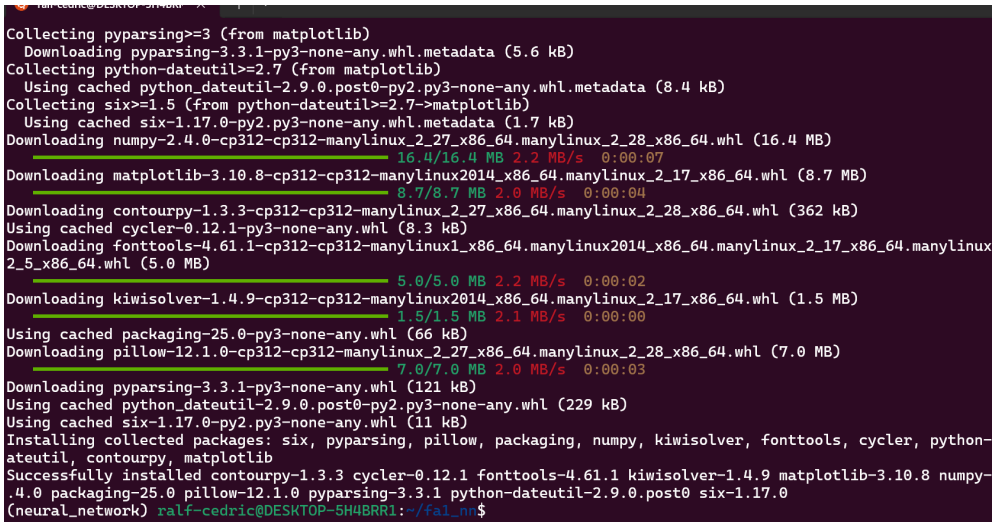
Before we install the necessary libraries in the environment, we need to update the package manager (pip) to the latest version. This is to avoid installation errors from outdated tools. Do this:

```
python3 -m pip install --upgrade pip
```



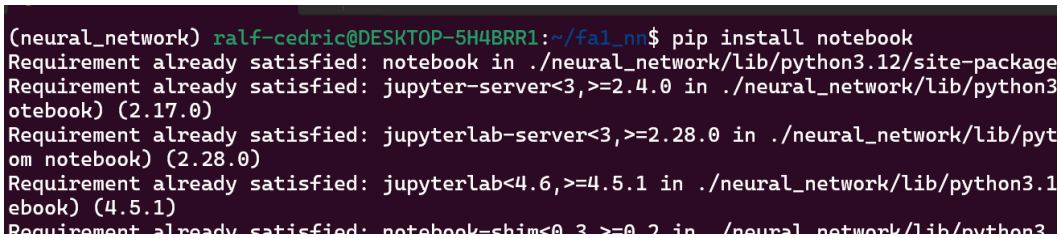

Most basic neural network repositories require basic libraries such as NumPy and Matplotlib. NumPy is renowned for its capabilities in scientific computing, which is utilized in neural networks for computing weights, activations, and gradients (NumPy, 2025). Matplotlib is known for its visualizations, which will help us present the results of the neural network, allowing us to understand its behavior and performance (Matplotlib, 2024).

```
python3 -m pip install numpy matplotlib
```



Next, we will install Jupyter Notebook in the environment to allow the display of the full output of the Neural Network implementation later:

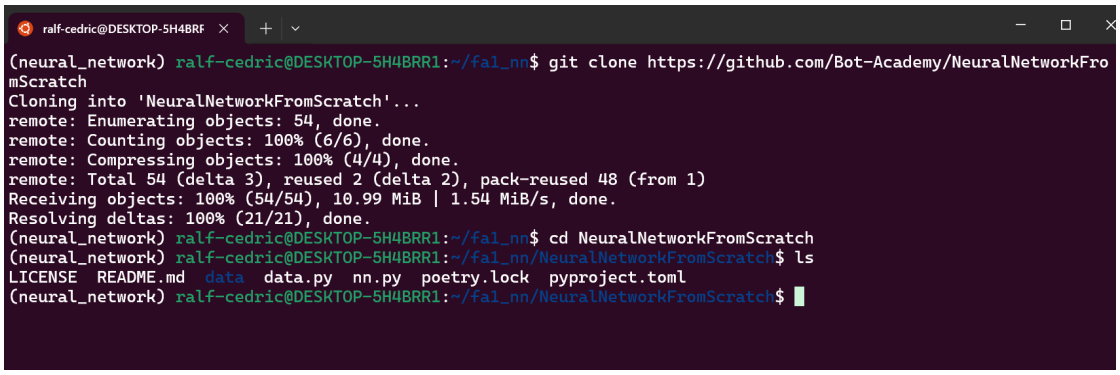
```
pip install notebook
```



# V. Getting the Repository from GitHub

Now, we will clone the sample code from GitHub as provided. Inside the same folder *(~/fa1_nn)*, run:

```
git clone https://github.com/Bot-Academy/NeuralNetworkFromScratch
cd NeuralNetworkFromScratch
ls
```




Once you are able to change directories *(`cd`)*, you have successfully cloned the sample code. You can also check the contents of the repository by listing them *(`ls`)*.

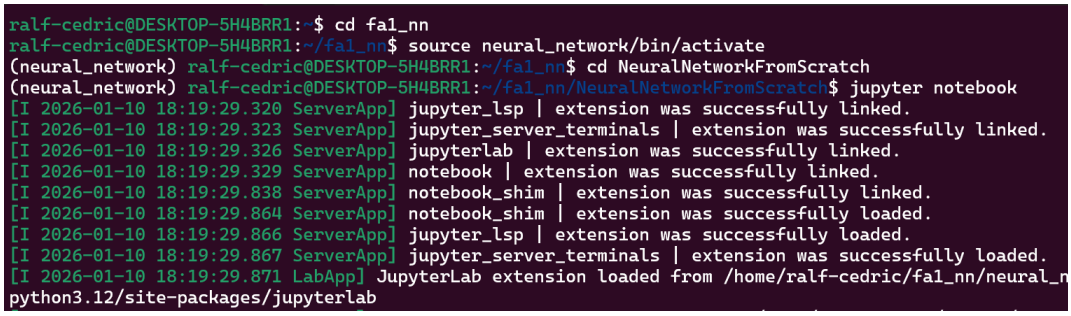
# VI. Running the Code and Its Output

From the file list, the main files are:
`nn.py`, `data.py`, `LICENSE`, `README.md`, `data`, `poetry.lock`, `pyproject.toml`

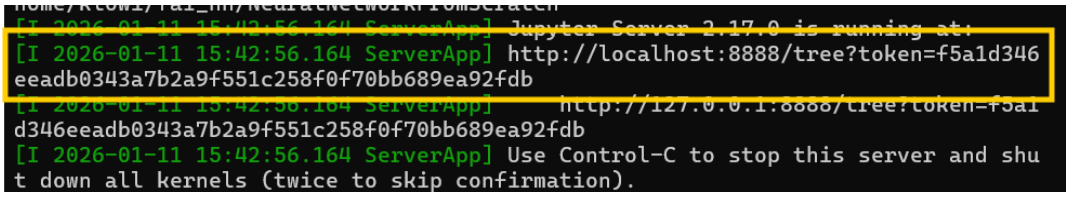
These two are essential for our neural network:

| | |
|---|---|
| `nn.py` | Neural Network Implementation |
| `data.py` | Data Handling |

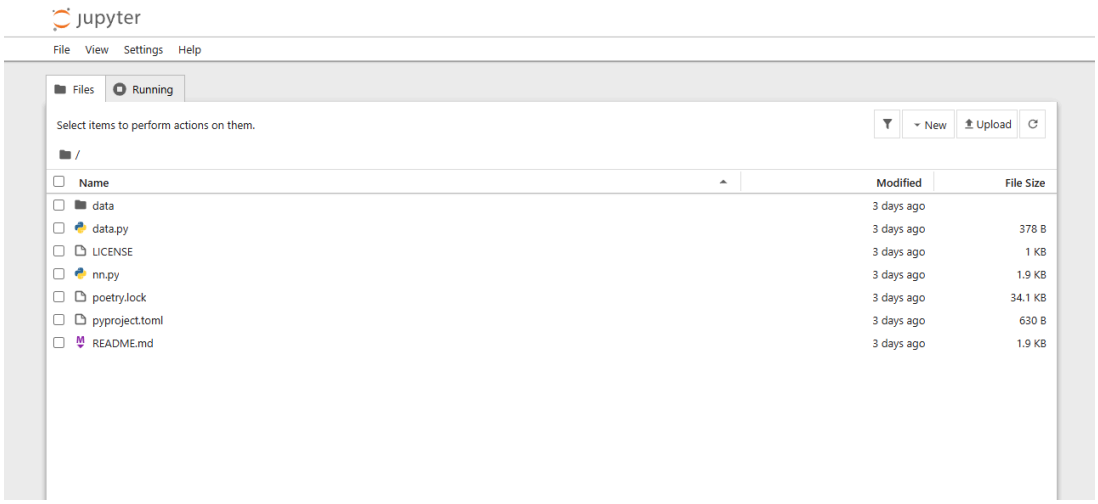
Now, open a Jupyter notebook to run the code in there. You can do this by typing `jupyter notebook`.




A link to open the Jupyter Notebook website will be shown in the terminal after inputting the command. To correctly identify the link, look for the one that has localhost:8888 with a token. Here is an example:




Copy and paste this link into your browser to open. Remember that you can always close this by *Ctrl + C*!




Once opened, the files can be seen here. Create a new `IPython notebook` where we will run `nn.py` by clicking the '`New`' at the top right and choosing `Python 3 (ipykernel)`. Rename it as you'd like.

In the first cell, copy all the code in the `nn.py`. While you can run the `.py` file, we are doing this so we can put this in an `.ipynb`. After this, run the cell:

```
[*]: from data import get_mnist
import numpy as np
import matplotlib.pyplot as plt

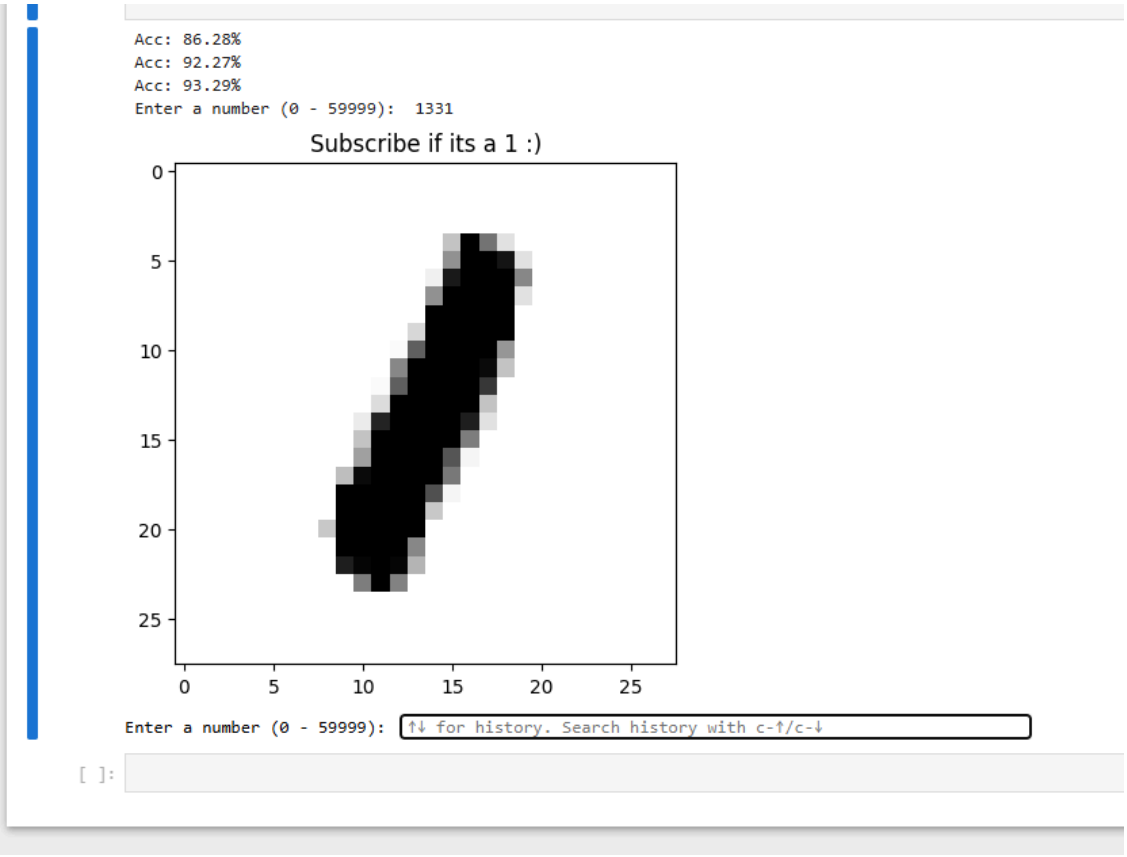

"""
w = weights, b = bias, i = input, h = hidden, o = output, l = label
e.g. w_i_h = weights from input layer to hidden layer
"""
images, labels = get_mnist()
w_i_h = np.random.uniform(-0.5, 0.5, (20, 784))
w_h_o = np.random.uniform(-0.5, 0.5, (10, 20))
b_i_h = np.zeros((20, 1))
b_h_o = np.zeros((10, 1))

learn_rate = 0.01
nr_correct = 0
epochs = 3
for epoch in range(epochs):
    for img, l in zip(images, labels):
        img.shape += (1,)
        l.shape += (1,)
        # Forward propagation input -> hidden
        h_pre = b_i_h + w_i_h @ img
        h = 1 / (1 + np.exp(-h_pre))
        # Forward propagation hidden -> output
        o_pre = b_h_o + w_h_o @ h
        o = 1 / (1 + np.exp(-o_pre))

        # Cost / Error calculation
        e = 1 / len(o) * np.sum((o - l) ** 2, axis=0)
        nr_correct += int(np.argmax(o) == np.argmax(l))

        # Backpropagation output -> hidden (cost function derivative)
        delta_o = o - l
        w_h_o += -learn_rate * delta_o @ np.transpose(h)
        b_h_o += -learn_rate * delta_o
        # Backpropagation hidden -> input (activation function derivative)
        delta_h = np.transpose(w_h_o) @ delta_o * (h * (1 - h))
```

After running, scroll down to the very bottom, and the accuracies are shown. When prompted for a number, enter **'1331'**, like so:




From a handwritten number '1', the model correctly identifies this number as the number "1".

# VII. Neural Network Discussion (Optional)

While this discussion may be optional, it is necessary to fully understand the Neural Network. We will try to uncover the basics of neural networks with this.

The first block of code loads the MNIST dataset (`get_mnist()`). This dataset contains a wide variety of handwritten digits used for training neural networks (GeeksforGeeks, 2024). This dataset contains 60,000 images that will be flattened into 784 pixels and labeled with 10 one-hot vectors for training.

This is followed by the creation of random weights (between -0.5 and 0.5) for a **20 × 784 matrix** (`w_i_h`) from the input to the hidden layer, resulting in **20 neurons (hidden) from 784 pixels (input).** This means that each of the 20 hidden neurons has 784 weights. Additionally, random weights from the hidden layer to the output layer are set to **10 × 20** (`w_h_o`), resulting in **20 neurons (hidden) that produce 10 outputs** (corresponding to digits 0-9). The biases (`b_h_o` & `b_i_h`) are also initialized to zero in their respective dimensions.

```
images, labels = get_mnist()
w_i_h = np.random.uniform(-0.5, 0.5, (20, 784))
w_h_o = np.random.uniform(-0.5, 0.5, (10, 20))
b_i_h = np.zeros((20, 1))
b_h_o = np.zeros((10, 1))
```

For the next code block, the learn rate is the step size when updating the weights and biases, while the `nr_correct` is the counter for accuracy. All in all, three (3) epochs will be done through the dataset for training.

```
learn_rate = 0.01
nr_correct = 0
epochs = 3
```

Inside each loop, it processes one image and its corresponding label, reshaping them into column vectors.

```
for img, l in zip(images, labels):
    img.shape += (1,)
    l.shape += (1,)
```

Now for the forward propagation (input → hidden), the weights are applied onto the image with matrix multiplication (`w_i_h @ img`), then bias is added (`+ b_i_h`). This calculates the **weighted sum of the hidden layer (`h_pre`)**. As for the next line, this is the **sigmoid function for the activation layer**. This calculates a value between 0 and 1 (`h`).

```
# Forward propagation input -> hidden
h_pre = b_i_h + w_i_h @ img
h = 1 / (1 + np.exp(-h_pre))
```

Next, for the forward propagation (hidden → output), the same thing is done. Weights are applied to the output of the hidden layer by matrix multiplication (`w_h_o @ h`). And a bias is added (`+ b_h_o`) to calculate the weighted sum (`o_pre`). The same activation layer is applied, which is Sigmoid. This results in a value between 0 and 1 again, but this time as the output (`o`).

```
# Forward propagation hidden -> output
o_pre = b_h_o + w_h_o @ h
o = 1 / (1 + np.exp(-o_pre))
```

The cost calculation in this code (`e`) refers to the mean squared error. This tells us how far the guess was from the actual label. The next line in the code checks for the number of accurately predicted digits from the true digit, which adds 1 if it is.

```
# Cost / Error calculation
e = 1 / len(o) * np.sum((o - l) ** 2, axis=0)
nr_correct += int(np.argmax(o) == np.argmax(l))
```

The backpropagation will be explained in the next section. For now, we will proceed with the rest of the code blocks.

```
    # Show accuracy for this epoch
    print(f"Acc: {round((nr_correct / images.shape[0]) * 100, 2)}%")
    nr_correct = 0
```

Once the entire dataset has been passed through, the accuracy for the current epoch is displayed, then nr_correct is initialized to 0 for a new epoch.

```
# Show results
while True:
    index = int(input("Enter a number (0 - 59999): "))
    img = images[index]
    plt.imshow(img.reshape(28, 28), cmap="Greys")

    img.shape += (1,)
    # Forward propagation input -> hidden
    h_pre = b_i_h + w_i_h @ img.reshape(784, 1)
    h = 1 / (1 + np.exp(-h_pre))
    # Forward propagation hidden -> output
    o_pre = b_h_o + w_h_o @ h
    o = 1 / (1 + np.exp(-o_pre))

    plt.title(f"Subscribe if its a {o.argmax()} :)")
    plt.show()
```

After all three (3) training loops are done, the code will proceed with the testing loop. This is the part where you pick an image, and the displayed image will be predicted by the model.

# VIII. Locate the Backpropagation of the Neural Network

Backpropagation is where neural networks learn from their mistakes, and occurs right after forward propagation and error computation. This allows the mechanism to improve its performance by minimizing prediction error. This is through calculating gradients and updating weights by propagating the error from the output layer backward using gradient descent (Goodfellow, Bengio, & Courville, 2016). The network, then, adjusts its weight and biases in terms of how far the prediction is from the true label, and how far each step is taken is determined by the learning rate.

As indicated in the code in the comments, the backpropagation in the neural network can be seen as such:

```
# Backpropagation output -> hidden (cost function derivative)
delta_o = o - l
w_h_o += -learn_rate * delta_o @ np.transpose(h)
b_h_o += -learn_rate * delta_o
```

For the output → hidden, we can observe that delta_o has been seen from the error, which calculates the **error at the output**. The following equations try to update the hidden → output weights (`w_h_o`) and the bias (`b_h_o`).

The error gradient `delta_o` is computed as the difference between the predicted output `o` and the true label `l`. This tells how much each output neuron contributes to the decision. Once the network computes this, the weights connecting the hidden layer to the output layer are updated using gradient descent; how much the weights change is dependent on the learning rate. Through countless updates, these steps make up for the network's final predictions (Nielsen, 2015).

```
# Backpropagation hidden -> input (activation function derivative)
delta_h = np.transpose(w_h_o) @ delta_o * (h * (1 - h))
w_i_h += -learn_rate * delta_h @ np.transpose(img)
b_i_h += -learn_rate * delta_h
```

As for the hidden → input, it is further propagated backward to the hidden layer, and the delta_h calculates the **error for the hidden layer**, with a derivative of the Sigmoid

function to aid in computing. Again, the following equations adjust the hidden → input weights (`w_i_h`) and biases (`b_i_h`). The term h * (1-h) is the derivative of the sigmoid activation function. This determines how sensitive each hidden neuron is to the error. Multiplying this derivative by the error from the output layer will let the network know how much each hidden neuron contributes to the overall error.

# IX. References


Git. (2025, November 17). *Git - git Documentation*. Git-Scm.com. https://git-scm.com/docs/git

GeeksforGeeks. (2024, May). MNIST Dataset: Practical Applications Using Keras and PyTorch. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/mnist-dataset/

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning. MIT Press.* https://www.deeplearningbook.org

Matplotlib. (2024). *Matplotlib: Python plotting — Matplotlib 3.3.4 documentation*. Matplotlib.org. https://matplotlib.org/stable/index.html

Nielsen, M. A. (2015). *Neural Networks and Deep Learning.* Matplotlib.org. http://neuralnetworksanddeeplearning.com

NumPy. (2025). *NumPy documentation — NumPy v2.4 Manual*. Numpy.org. https://numpy.org/doc/2.4/

pip. (2008). *pip - The Python Package Installer — pip 19.0.3 documentation*. Pypa.io. https://pip.pypa.io/en/stable/

Python Software Foundation. (2019). *venv — Creation of virtual environments — Python 3.8.1 documentation*. Python.org. https://docs.python.org/3/library/venv.html